SC1007 Data Structures & Algorithms
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Arrays, Linked Lists & Foundations

> *"The machine can only store bits contiguously or linked by pointers."* — Every data structure trades off these two extremes.

Prerequisites: SC1003 (C/Python basics). This chapter is the 0→1: how data lives in memory and how STL wraps it.

Learning Outcomes

After this chapter you will be able to:

(L1) compare array vs. linked-list layouts and their $O(\cdot)$ costs;

(L2) implement singly/doubly linked lists with invariants;

(L3) analyse dynamic arrays (amortised doubling) and use STL `vector`;

(L4) choose the right sequential container and justify it.

1.1 Contiguous Storage: Arrays

A *pointer* is a variable whose value is a memory address; dereferencing follows the address to the stored object. Asymptotic costs: $f(n) = \Theta(g(n))$ means $c_1 g(n) \le f(n) \le c_2 g(n)$ for large n (tight bound), $O(g)$ is an upper bound, $\Omega(g)$ a lower bound. $O(1)$ = constant time independent of n; $O(n)$ = linear in n.

An *array* stores n elements consecutively. If base address is B and each element occupies w bytes,

$$\text{addr}(A[i]) = B + i \cdot w, \tag{1.1}$$

so random access is $\Theta(1)$. Insertion/deletion at position k shifts $n - k$ elements: $\Theta(n)$.

Static vs. dynamic. Static arrays have fixed capacity. A *dynamic array* (C++ `vector`, Java `ArrayList`, Python `list`) doubles capacity when full. Appending n items costs $O(n)$ total — amortised $O(1)$ per append.

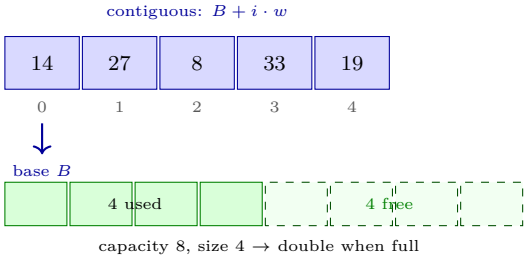

Figure 1.1: Array in memory (top): index → address in $O(1)$. Dynamic array (bottom): doubling gives amortised $O(1)$ append.

```cpp
// C++ STL vector -- dynamic array
#include <vector>
std::vector<int> v = {14, 27, 8};
v.push_back(33);              // amortised O(1)
int x = v[1];                 // O(1) random access
v.insert(v.begin()+1, 99);  // O(n) -- shifts right
```

1.2 Linked Storage: Linked Lists

A *linked list* scatters nodes; each node holds data plus pointer(s) to neighbours. No shifting is needed for mid-list insertion.

Definition 1.1 (Singly linked list)**.** A chain $p_0 \to p_1 \to \cdots \to p_{n-1} \to \texttt{null}$ where node p_i stores $(x_i, \texttt{next})$. Doubly linked adds `prev`.

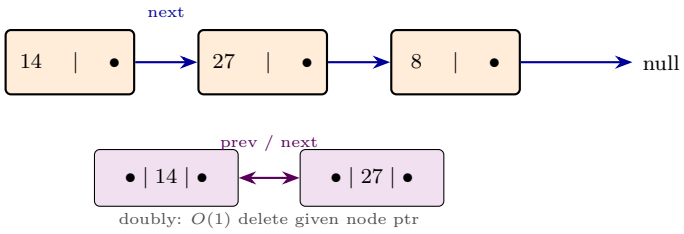

Figure 1.2: Singly linked list (top) vs. doubly linked fragment (bottom). Pointer chasing replaces index arithmetic.

Operations: access by index $\Theta(n)$ (must walk); insert/delete at head $\Theta(1)$; after locating position k, pointer rewiring is $\Theta(1)$. Doubly linked allows $O(1)$ deletion given a node pointer.

```python
# Linked list node -- Python
class Node:
    def __init__(self, val, nxt=None):
        self.val = val
        self.next = nxt

def insert_after(node, val):
    node.next = Node(val, node.next)  # O(1)

def delete_after(node):
```

```
    if node.next:
        node.next = node.next.next    # O(1)
```

1.3 Complexity Comparison & STL Choice

Operation	Array / `vector`	Linked list
Access $A[k]$	$\Theta(1)$	$\Theta(n)$
Append at end	amort. $\Theta(1)$	$\Theta(1)^*$
Insert at k	$\Theta(n-k)$ shift	$\Theta(k)$ seek + $\Theta(1)$ link
Delete at k	$\Theta(n-k)$ shift	$\Theta(k)$ seek + $\Theta(1)$ link
Memory overhead	low (contiguous)	+1–2 ptrs / elem

Table 1.1: Sequential containers compared. *With tail pointer; else $\Theta(n)$.

Rule of thumb: prefer `vector` unless you need frequent mid-list splicing with iterators or non-contiguous growth without reallocation.

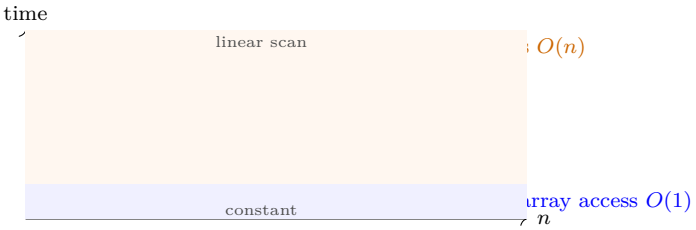

Figure 1.3: Random access: array stays flat; linked list grows linearly with n.

1.4 Iterators & Amortised Doubling

STL decouples containers from algorithms via *iterators*. A vector iterator is a random-access pointer; a list iterator is bidirectional. Algorithms like `std::sort` require random access, so they work on vectors but not lists.

Amortised doubling proof. Start with capacity 1. Insert n items, doubling when full. Total copies:

$$1 + 2 + 4 + \cdots + 2^{\lceil \log_2 n \rceil - 1} < 2n. \tag{1.2}$$

So n appends cost $O(n)$ and average $O(1)$. Halving when size $<$ capacity/4 keeps both push and pop amortised $O(1)$ without thrashing.

```
// Iterator example -- same code works for vector and deque
#include <vector>
#include <algorithm>
std::vector<int> v = {5,2,9,1,7};
std::sort(v.begin(), v.end()); // requires random-access iterators
for (auto it = v.begin(); it != v.end(); ++it) *it += 1;
```

Remark 1.1. Python `list` over-allocates similarly; CPython grows by $\approx 1.125\times$ for large lists. Java `ArrayList` grows by $1.5\times$. Constants affect memory but not the $O(1)$ amortised bound.

1.5 Chapter Notes

CLRS Ch. 10, Weiss Ch. 3. STL: `vector` is dynamic array; `list` is doubly linked; `deque` is in Ch. 2. Python `list` is a dynamic array, not a linked list.

1.6 Exercises

E1.1 **Address arithmetic.** Array `int A[100]` at base $B = 0x1000$, $w = 4$ bytes. (a) Address of `A[37]`? (b) Why does `A[i][j]` in $m \times n$ row-major cost $B + (i \cdot n + j)w$?

E1.2 **Doubling analysis.** Starting from capacity 1, insert $n = 2^k$ items with doubling. Count total element copies. Show it is $< 2n$ and deduce amortised $O(1)$.

E1.3 **Linked list reversal.** Write iterative $O(n)$-time $O(1)$-space reversal for singly linked list. Prove your loop invariant: after t iterations the prefix of length t is reversed.

E1.4 **Container choice.** For each scenario choose `vector` vs. `list` and justify: (a) 1M insertions at random positions with iterator kept; (b) binary search over sorted data; (c) queue with push/pop at ends only.

E1.5 **Splice.** Given doubly linked list with head/tail, splice node x (in L_1) into L_2 after node y in $O(1)$. Give pointer updates and handle head/tail edge cases.

Chapter 2

Stacks, Queues & Deques

> *"Push, pop, enqueue, dequeue."* — Four operations, yet they power recursion, BFS, undo, and scheduling.

Prerequisites: Ch. 1 (arrays/lists, $O(\cdot)$). This chapter adds restricted-access structures and classic applications.

Learning Outcomes

After this chapter you will be able to:

(L1) implement stacks/queues/deques via arrays and linked lists in $O(1)$;

(L2) use stacks for matching, postfix evaluation, and recursion simulation;

(L3) use queues for BFS preview and scheduling; explain circular buffers;

(L4) choose among stack / queue / deque and analyse amortised costs.

2.1 Stacks: LIFO

A *stack* is LIFO: `push` adds at the top, `pop` removes from the top, `peek` inspects. All $\Theta(1)$.

```python
# Stack via Python list -- amortised O(1) push/pop
stack = []
stack.append(33)    # push
stack.append(27)
x = stack.pop()     # pop -> 27
top = stack[-1]     # peek -> 33
```

Applications: balanced brackets (push open, pop on close), postfix evaluation ($3\,4\,+ \rightarrow$ push 3, push 4, pop twice, push 7), call stack (each call pushes a frame), DFS via explicit stack.

```c
/* Balanced brackets -- C */
```

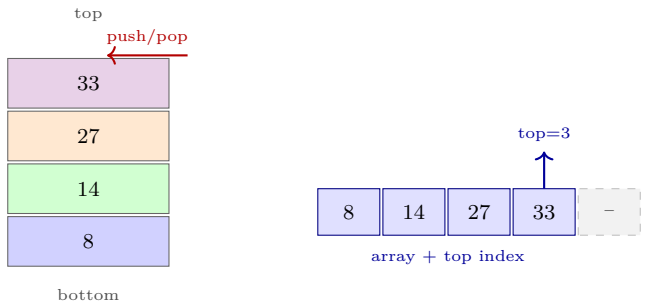

Figure 2.1: Stack plates (LIFO) and array implementation with `top` pointer. Linked-list version uses head as top.

```
int balanced(const char *s) {
    char st[1000]; int top = -1;
    for (int i = 0; s[i]; i++) {
        if (s[i]=='(' || s[i]=='[') st[++top] = s[i];
        else if (s[i]==')') { if (top<0||st[top--]!='(') return 0; }
        else if (s[i]==']') { if (top<0||st[top--]!='[') return 0; }
    }
    return top == -1;
}
```

2.2 Queues: FIFO

A *queue* is FIFO: `enqueue` at rear, `dequeue` from front, both $\Theta(1)$ with a linked list (head/tail) or a *circular buffer* over an array.

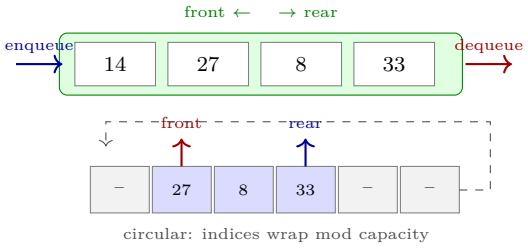

Figure 2.2: Queue (FIFO) and circular-buffer array implementation with `front`/`rear` wrapping.

Circular buffer: store `front`, `size`, `cap`; `rear` = (`front` + `size`) mod `cap`. Enqueue/dequeue adjust pointers modulo `cap`; no shifting.

2.3 Deques

A *deque* (double-ended queue) allows push/pop at *both* ends in $\Theta(1)$. Implemented by doubly linked list or a circular buffer with two pointers. C++ `deque` gives $O(1)$ random access plus $O(1)$ ends.

```
from collections import deque
dq = deque([27, 8])
dq.appendleft(14)   # O(1) left push
```

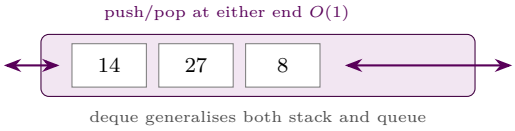

Figure 2.3: Deque supports operations at both ends, subsuming stack and queue.

```
dq.append(33)        # O(1) right push
dq.popleft()         # O(1) -> 14
dq.pop()             # O(1) -> 33
```

Use cases: sliding-window maximum (monotone deque, Ch. 8), work stealing, palindrome checking, undo/redo with two stacks.

2.4 Implementations Compared

Structure	Array	Linked list	Circular buf.
Stack push/pop	$O(1)$	$O(1)$	—
Queue enq/deq	$O(n)$ shift	$O(1)$	$O(1)$
Deque both ends	$O(n)$	$O(1)$	$O(1)$
Cache locality	excellent	poor	excellent

Table 2.1: Implementation trade-offs. Circular buffer wins for queue/deque on arrays.

2.5 Applications: Matching, Evaluation, BFS Preview

Postfix evaluation. Scan tokens left to right: push operands; on operator pop two, apply, push result. Example $3\,4\,+\,2\,\times = 14$. Infix $\to$ postfix via shunting-yard (operator stack) handles precedence.

Function calls. The call stack stores frames (locals, return address). Recursion depth = stack height; tail recursion can be optimised to iteration.

BFS preview. A queue gives level-order traversal (Ch. 3) and shortest paths in unweighted graphs (Ch. 7). Replacing the queue with a stack yields DFS — same code, different data structure, radically different order.

```
# Postfix evaluator -- O(n)
def eval_postfix(tokens):
    st = []
    for tok in tokens:
        if tok in "+-*/":
            b, a = st.pop(), st.pop()
            st.append(str(eval(f"{a}{tok}{b}")))
        else:
            st.append(tok)
    return int(st[0])
# eval_postfix(["3","4","+","2","*"]) == 14
```

Remark 2.1. Monotone deque for sliding-window maximum: maintain decreasing deque of candidates; each element enters and leaves once — $O(n)$ for window size k over n elements, previewing Ch. 8.

2.6 Chapter Notes

CLRS Ch. 10.1, Weiss Ch. 3.3–3.6. STL: `stack` and `queue` are adapters over `deque` by default; `priority_queue` is in Ch. 5.

2.7 Exercises

E2.1 **Circular queue.** Implement queue with array `A[0..cap-1]`, integers `front`, `sz`. Give `enqueue`, `dequeue`, `isEmpty`, `isFull` in $O(1)$. How do you distinguish full vs. empty?

E2.2 **Two stacks, one array.** Use a single array of size n for two stacks growing from opposite ends. Give push/pop for each and prove overflow iff total items $> n$.

E2.3 **Postfix evaluation.** Evaluate $5\,1\,2\,+\,4\,*\,+\,3\,-$ with a stack, showing contents after each token. Write $O(n)$ algorithm for general postfix over $+, -, \times, /$.

E2.4 **Queue via stacks.** Implement queue using two stacks with amortised $O(1)$ enqueue/dequeue. Prove amortised bound via potential $\Phi =$ size of input stack.

E2.5 **Deque min.** Design deque supporting `pushFront/Back`, `popFront/Back`, and `getMin` in $O(1)$ amortised (hint: auxiliary monotone deque).

Chapter 3

Trees & Traversals

> *"A tree is a hierarchy you can walk without getting lost."* — Traversals turn branching structure into linear order.

Prerequisites: Ch. 1–2 (pointers, recursion, stack/queue). This chapter is the foundation for all search trees (Ch. 4–5) and graph traversals (Ch. 7).

Learning Outcomes

After this chapter you will be able to:

(L1) define trees, binary trees, height/depth and prove n nodes $\to n-1$ edges;

(L2) implement the three DFS traversals (pre/in/postorder) recursively and iteratively;

(L3) perform level-order (BFS) traversal and relate it to queues;

(L4) reconstruct a binary tree from traversal pairs and analyse $O(n)$ costs.

3.1 Tree Terminology

A *tree* is a connected acyclic graph. A *rooted tree* designates one node as root; every other node has a unique parent. A *binary tree* gives each node at most two children: `left` and `right` (order matters).

For node v: *depth* = edges from root to v; *height* = longest downward path to a leaf. Height of tree = height of root. Height of single-node tree = 0; empty tree = -1 by convention.

Theorem 3.1. A tree with n nodes has exactly $n-1$ edges. A binary tree of height h has at most $2^{h+1}-1$ nodes.

Storage: each node holds `val`, `left`, `right` pointers. Array embedding (heap-style, Ch. 5) stores node i's children at $2i, 2i+1$.

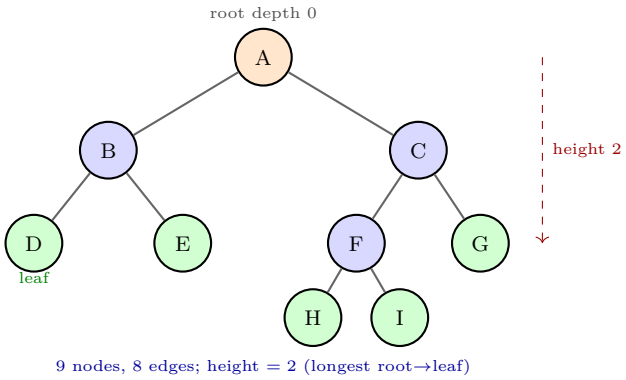

Figure 3.1: A rooted binary tree. Orange = root, blue = internal, green = leaves. Depth and height illustrated.

3.2 Depth-First Traversals

DFS visits recursively; three orders differ only in when the root is processed:

$$\begin{aligned} \text{Preorder: } & \text{root} \to \text{left} \to \text{right} \\ \text{Inorder: } & \text{left} \to \text{root} \to \text{right} \\ \text{Postorder: } & \text{left} \to \text{right} \to \text{root} \end{aligned}$$

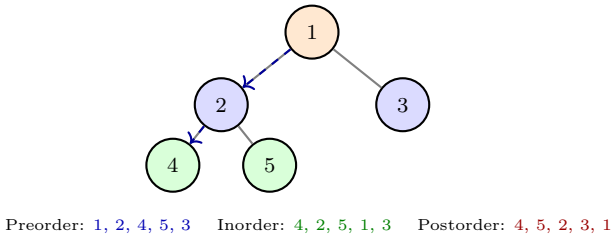

Figure 3.2: Five-node example with all three DFS orders. Preorder is used to copy; postorder to delete; inorder gives sorted order for BSTs (Ch. 4).

```python
# Recursive traversals -- O(n) time, O(h) stack space
def preorder(root):
    if not root: return
    visit(root)              # root first
    preorder(root.left)
    preorder(root.right)

def inorder(root):
    if not root: return
    inorder(root.left)
    visit(root)              # root in middle
    inorder(root.right)

def postorder(root):
    if not root: return
    postorder(root.left)
    postorder(root.right)
    visit(root)              # root last
```

```
19
20 # Iterative preorder with explicit stack
21 def preorder_iter(root):
22     stack = [root] if root else []
23     while stack:
24         v = stack.pop()
25         visit(v)
26         if v.right: stack.append(v.right)
27         if v.left:  stack.append(v.left)
```

Each node is visited once, each edge traversed twice (down and up): $\Theta(n)$ time, $\Theta(h)$ stack space ($\Theta(n)$ worst case for a chain, $\Theta(\log n)$ for balanced).

3.3 Level Order (BFS)

Level order visits nodes breadth-first using a queue:

```
1 from collections import deque
2 def level_order(root):
3     if not root: return
4     q = deque([root])
5     while q:
6         v = q.popleft()
7         visit(v)
8         if v.left:  q.append(v.left)
9         if v.right: q.append(v.right)
```

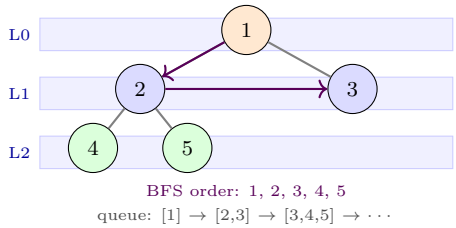

Figure 3.3: Level-order traversal processes level d before $d+1$. Queue holds the frontier.

3.4 Reconstruction

Given inorder + preorder (or inorder + postorder) the tree is uniquely determined. Preorder gives root first; inorder splits left/right subtrees. $O(n)$ with a hash map from value to inorder index. Preorder + postorder alone is ambiguous for general binary trees.

3.5 Tree Properties & Variations

Full / complete / perfect. Full: every node has 0 or 2 children. Complete: all levels full except last, filled left-to-right (heap shape, Ch. 5). Perfect: all leaves at same depth, $n = 2^{h+1} - 1$.

Threaded trees. Replace null child pointers with threads to inorder successor/predecessor, enabling $O(1)$-space inorder without stack.

Expression trees. Leaves are operands, internal nodes operators. Inorder with parentheses recovers infix; preorder gives prefix (Polish); postorder gives postfix — directly linking traversals to Ch. 2 evaluation.

```
# Iterative inorder -- O(n) time, O(h) space, no recursion
def inorder_iter(root):
    stack, cur, out = [], root, []
    while stack or cur:
        while cur:
            stack.append(cur); cur = cur.left
        cur = stack.pop()
        out.append(cur.val)
        cur = cur.right
    return out
```

3.6 Chapter Notes

CLRS Ch. 10.4, 12. Weiss Ch. 4. DFS $\leftrightarrow$ stack, BFS $\leftrightarrow$ queue duality previews Ch. 7 graphs.

3.7 Exercises

E3.1 **Orders.** For the tree in Fig. 3.1, list preorder, inorder, postorder, and level order.

E3.2 **Iterative inorder.** Give an $O(n)$ iterative inorder using one stack (no recursion). Prove it visits nodes in inorder and uses $O(h)$ space.

E3.3 **Reconstruction.** Preorder $= [3, 9, 20, 15, 7]$, inorder $= [9, 3, 15, 20, 7]$. Reconstruct the tree and give its postorder. Explain the $O(n)$ hash-map algorithm.

E3.4 **Height vs. nodes.** Prove: a binary tree with n nodes has height $h \geq \lceil \log_2(n+1) \rceil - 1$, with equality for perfect trees. Give an example where $h = n - 1$.

E3.5 **Expression tree.** Build the tree for $(3 + 4) \times (5 - 2)$ and show that preorder/postorder/inorder give prefix/postfix/infix notation. Which needs parentheses?

Chapter 4

Binary Search Trees & Balanced Trees

"A sorted tree is a sorted array you can grow." — But only if it stays balanced.

Prerequisites: Ch. 3 (trees, traversals). BST is the bridge from trees to ordered dictionaries; AVL/red-black keep it $O(\log n)$.

Learning Outcomes

After this chapter you will be able to:

(L1) maintain the BST invariant and implement search/insert/delete in $O(h)$;

(L2) explain why worst-case $h = \Theta(n)$ and how balancing fixes it;

(L3) perform AVL rotations and verify the AVL balance invariant;

(L4) sketch red-black invariants and argue $O(\log n)$ height.

4.1 Binary Search Trees

Definition 4.1 (BST invariant)**.** For every node v, all keys in v.left are $< v$.key and all keys in v.right are $> v$.key (assume distinct keys).

Consequence: inorder traversal yields keys in sorted order (Ch. 3).

Operations (all $O(h)$ where h is height):

```
def search(root, k):
    while root and root.key != k:
        root = root.left if k < root.key else root.right
    return root  # None if absent

def insert(root, k):
    if not root: return Node(k)
```

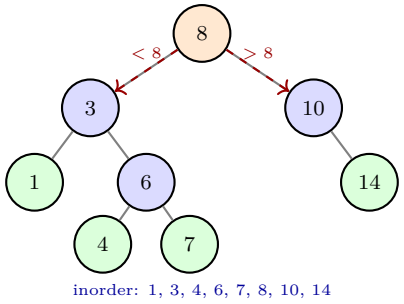

Figure 4.1: A BST. Every left subtree holds smaller keys, every right subtree larger. Inorder is sorted.

```
    if k < root.key:  root.left  = insert(root.left, k)
    elif k > root.key: root.right = insert(root.right, k)
    return root

def delete(root, k):  # three cases: leaf / one child / two children
    if not root: return None
    if k < root.key:  root.left  = delete(root.left, k)
    elif k > root.key: root.right = delete(root.right, k)
    else:  # found
        if not root.left:  return root.right
        if not root.right: return root.left
        # two children: replace with successor
        succ = min_node(root.right)
        root.key = succ.key
        root.right = delete(root.right, succ.key)
    return root
```

Successor = minimum in right subtree (leftmost descendant). All ops follow one root-to-leaf path: $O(h)$ time, $O(h)$ stack space.

Degeneracy. Inserting sorted keys $1, 2, 3, \ldots, n$ yields a chain with $h = n - 1$: every operation degrades to $\Theta(n)$. Balanced trees prevent this.

4.2 AVL Trees

AVL (Adelson-Velsky–Landis, 1962) maintains the *AVL invariant*: for every node,

$$|\,\text{height}(\text{left}) - \text{height}(\text{right})\,| \le 1. \tag{4.1}$$

This forces $h = \Theta(\log n)$ (Fibonacci bound: $n \ge F_{h+2} - 1$, so $h \le 1.44 \log_2 n$).

Rebalancing via rotations. After insertion/deletion, walk up updating heights; at the first unbalanced node, apply one of four rotations:

Insert/delete may need $O(\log n)$ rotations walking to root, but each rotation is $O(1)$.

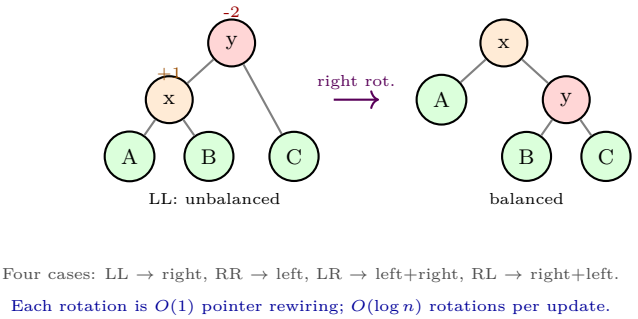

Figure 4.2: AVL right rotation for the LL case (left child too tall). The other three cases are symmetric. After rotation, heights of x, y are recomputed and the invariant is restored.

Figure 4.3: Height comparison: unbalanced BST can be linear; AVL guarantees logarithmic height.

4.3 Red-Black Trees (Sketch)

Red-black trees relax the AVL condition for fewer rotations. Each node is coloured red or black with invariants: (i) root black, (ii) no two reds adjacent, (iii) every root-to-leaf path has same number of black nodes (black-height). These imply $h \leq 2\log_2(n+1)$.

Insertion colours the new node red, then fixes violations by recolouring and at most two rotations along the path. STL `map`/`set` (C++) and Java `TreeMap` are typically red-black trees.

AVL is more strictly balanced (faster lookups); red-black has fewer rotations on updates (faster insertions). Both are $O(\log n)$ worst-case.

4.4 Analysis & Rotation Details

Search cost $= h+1$ comparisons. In a random BST built from random insertions, expected $h = \Theta(\log n)$; worst-case $h = n-1$ (sorted input). This motivates self-balancing.

Rotations preserve inorder. A right rotation at y with left child x:

$$\text{inorder}(y \text{ subtree before}) = A, x, B, y, C = \text{inorder}(x \text{ subtree after}), \tag{4.2}$$

where A, B, C are subtrees. Only $O(1)$ pointers change; heights of x, y are recomputed. Left rotation is symmetric; LR = left on x then right on y.

```
# Right rotation at y (y.left == x) -- O(1)
```

```
def rotate_right(y):
    x = y.left
    y.left = x.right
    x.right = y
    update_height(y); update_height(x)
    return x  # new subtree root
```

Remark 4.1. Red-black insertion needs at most 2 rotations; AVL may need $O(\log n)$ up the path. Hence red-black is preferred when insertions dominate, AVL when lookups dominate.

4.5 Chapter Notes

CLRS Ch. 12–13, Weiss Ch. 4.2–4.3. AVL Fibonacci bound proof by induction on height.

4.6 Exercises

E4.1 **BST ops.** Insert $8, 3, 10, 1, 6, 14, 4, 7$ into empty BST in order. Draw the tree after each insertion and list inorder. Delete 3 (two children) and show the successor replacement.

E4.2 **AVL insertion.** Insert $1, 2, 3, 4, 5, 6$ into empty AVL. Show rotations at each step and verify Eq. (4.1) holds.

E4.3 **Height bound.** Prove $n \geq F_{h+2} - 1$ for AVL of height h (F_k Fibonacci). Deduce $h \leq 1.44 \log_2 n$.

E4.4 **Red-black.** Insert $10, 20, 30, 15$ into empty red-black tree. Show colourings and rotations. Verify all three invariants after each insert.

E4.5 **Comparison.** Give a sequence where AVL does asymptotically more rotations than red-black on the same insertions, and explain why.

Chapter 5

Heaps & Priority Queues

"The smallest is always on top." — A heap trades total order for instant access to the extremum.

Prerequisites: Ch. 3 (trees), Ch. 1 (arrays). Heap is an array-embedded tree giving $O(\log n)$ priority-queue ops and $O(n \log n)$ sorting.

Learning Outcomes

After this chapter you will be able to:

(L1) state the heap invariant and map heap indices to tree positions;

(L2) implement `push`, `pop`, `heapify` with sift-up/down;

(L3) build a heap in $O(n)$ via bottom-up heapify and prove it;

(L4) use priority queues for scheduling and preview heap sort / Dijkstra.

5.1 The Heap Invariant

A (min-)heap is a *complete* binary tree where every node $\leq$ its children. The minimum is at the root. Completeness means all levels are full except possibly the last, filled left-to-right — so the tree maps perfectly to an array.

$$\text{For } i \text{ (1-indexed): } \text{parent}(i) = \lfloor i/2 \rfloor,\ \text{left}(i) = 2i,\ \text{right}(i) = 2i + 1. \tag{5.1}$$

With 0-indexed arrays: parent $(i-1)/2$, children $2i+1, 2i+2$.

Not a BST: heap only orders parent vs. child, not left vs. right. Inorder is not sorted.

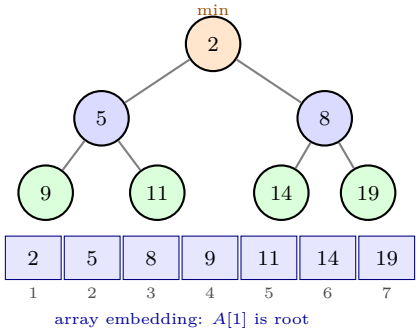

Figure 5.1: Min-heap as tree (top) and array (bottom). Every parent $\leq$ children; completeness gives $O(1)$ index arithmetic.

5.2 Priority-Queue Operations

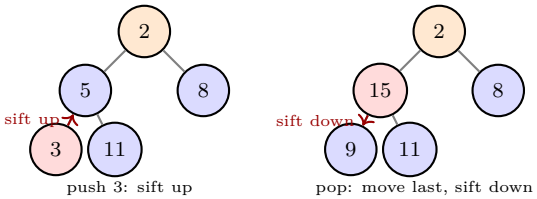

Figure 5.2: Sift-up after insertion (left) and sift-down after extraction (right). Each follows one root-to-leaf path: $O(\log n)$.

```python
import heapq  # Python min-heap (0-indexed)

def push(heap, x):
    heap.append(x)
    i = len(heap) - 1
    while i > 0:
        p = (i - 1) // 2
        if heap[p] <= heap[i]: break
        heap[p], heap[i] = heap[i], heap[p]
        i = p  # sift up -- O(log n)

def pop(heap):
    if len(heap) == 1: return heap.pop()
    top = heap[0]
    heap[0] = heap.pop()  # move last to root
    i = 0
    while True:  # sift down -- O(log n)
        l, r = 2*i+1, 2*i+2
        smallest = i
        if l < len(heap) and heap[l] < heap[smallest]: smallest = l
        if r < len(heap) and heap[r] < heap[smallest]: smallest = r
        if smallest == i: break
        heap[i], heap[smallest] = heap[smallest], heap[i]
        i = smallest
    return top
```

`peek` = `heap[0]` in $O(1)$. `decreaseKey` at index i sifts up in $O(\log n)$.

5.3 Building a Heap in $O(n)$

Naively pushing n items costs $O(n \log n)$. Bottom-up `heapify` does it in $O(n)$: process nodes from $n/2$ down to 1, sifting each down. Shorter subtrees dominate the sum.

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O\left(n \sum_{h \geq 0} h/2^h \right) = O(n), \tag{5.2}$$

since $\sum h/2^h = 2$.

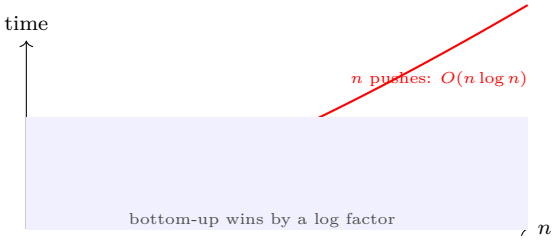

Figure 5.3: Heap construction: repeated push vs. bottom-up heapify.

```cpp
// C++ STL priority queue (max-heap by default)
#include <queue>
std::priority_queue<int> pq;          // max-heap
pq.push(5); pq.push(2); pq.push(8);
int top = pq.top();  // 8
pq.pop();            // removes 8

// min-heap variant
std::priority_queue<int, std::vector<int>, std::greater<int>> minpq;
```

5.4 Applications Preview

Heap sort: build heap $O(n)$, repeatedly pop min $O(n \log n)$ in-place. Dijkstra's algorithm (Ch. 7) uses a min-heap to extract the closest unsettled vertex in $O((n+m) \log n)$.

5.5 Variants & Analysis Details

Max-heap vs. min-heap. Flip comparison. Max-heap gives descending sort by repeated extraction; min-heap is natural for Dijkstra (Ch. 7).

d-ary heap. Each node has d children; height $\log_d n$, sift-down compares d children. Choose d to trade off sift-up vs. sift-down for the workload.

Decrease-key. After lowering $A[i]$, sift up restores heap in $O(\log n)$ — essential for Dijkstra and Prim.

```python
# heapq is min-heap; max-heap via negation
import heapq
```

```
max_heap = []
for x in [5,2,8,1]: heapq.heappush(max_heap, -x)
top = -heapq.heappop(max_heap)  # 8

# Decrease-key at index i (min-heap)
def decrease_key(heap, i, new_val):
    assert new_val <= heap[i]
    heap[i] = new_val
    while i > 0:
        p = (i-1)//2
        if heap[p] <= heap[i]: break
        heap[p], heap[i] = heap[i], heap[p]
        i = p
```

5.6 Chapter Notes

CLRS Ch. 6, Weiss Ch. 6. Heap as priority queue vs. sorted array vs. balanced BST trade-offs.

5.7 Exercises

E5.1 **Heap ops.** Insert $4, 8, 2, 9, 5, 11, 14, 3$ into empty min-heap (array order). Show array after each insert and after two pops.

E5.2 **Build-heap proof.** Prove Eq. (5.2) sums to $O(n)$ by bounding $\sum h/2^h$. Why does sifting from $n/2$ down to 1 suffice (leaves need no work)?

E5.3 **Heap vs. BST.** Give an array that is a valid min-heap but whose inorder is not sorted. Explain why heap search for arbitrary key is $\Theta(n)$.

E5.4 **k-way merge.** Merge k sorted arrays of total size n using a heap in $O(n \log k)$. Prove your bound and show it beats pairwise merging.

E5.5 **Heap sort in-place.** Describe in-place heap sort on array $A[1..n]$ (build max-heap, swap max to end, shrink). Analyse space $O(1)$ auxiliary and stability.

Chapter 6

Hashing

"Hashing is the closest we get to $O(1)$ search." — If the hash behaves, the dictionary flies.

Prerequisites: Ch. 1 (arrays), Ch. 5 (analysis). Hashing gives expected $O(1)$ insert/search/delete with high probability.

Learning Outcomes

After this chapter you will be able to:

(L1) design hash functions and explain universal hashing;

(L2) implement chaining and analyse $O(1+\alpha)$ expected cost;

(L3) implement open addressing (linear/quadratic/double hashing) and analyse clustering;

(L4) handle load factor, rehashing, and deletion correctly.

6.1 Hash Functions

A hash function $h : U \to \{0, \ldots, m-1\}$ maps universe U into m slots. Ideal: uniform, deterministic, fast. Two classic constructions for integer keys:

$$h(k) = k \bmod m \quad \text{(division, } m \text{ prime away from powers of 2)},$$
$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor \quad \text{(multiplication, } 0 < A < 1),$$
$$h(k) = ((ak + b) \bmod p) \bmod m \quad \text{(universal family, } p \text{ prime} > |U|).$$

Universal hashing picks a, b randomly at runtime, guaranteeing low collision probability for any adversarial input set.

Strings: polynomial rolling hash $h(s) = \sum_i s_i \cdot B^i \bmod m$ or Python's salted hash.

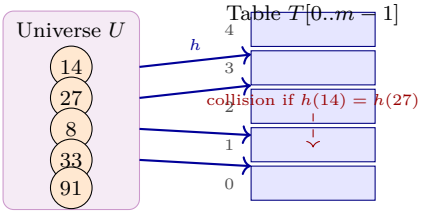

Figure 6.1: Hash function maps many keys into m slots. Collisions are inevitable when $|U| > m$ (pigeonhole) and must be resolved.

6.2 Chaining

Each slot $T[j]$ holds a linked list (chain) of keys hashing to j. Load factor $\alpha = n/m$.

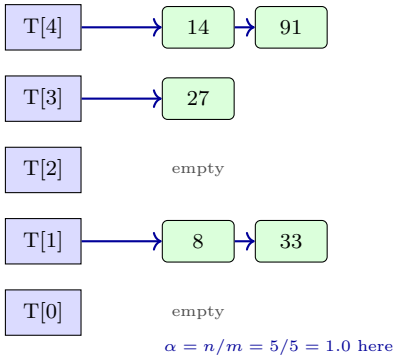

Figure 6.2: Chaining: each slot heads a linked list. Expected chain length = α; search scans the chain.

Under *simple uniform hashing*, expected search cost is $\Theta(1 + \alpha)$ (hash + chain scan). If $m = \Theta(n)$ then $\alpha = O(1)$ and operations are expected $O(1)$. Worst case $\Theta(n)$ if all keys collide, but universal hashing makes this vanishingly unlikely.

```python
# Chaining -- Python sketch
class ChainedHash:
    def __init__(self, m=8):
        self.m = m
        self.T = [[] for _ in range(m)]
    def _h(self, k): return hash(k) % self.m
    def insert(self, k, v):
        b = self._h(k)
        for i,(kk,_) in enumerate(self.T[b]):
            if kk == k: self.T[b][i]=(k,v); return
        self.T[b].append((k,v))
    def search(self, k):
        for kk,v in self.T[self._h(k)]:
            if kk == k: return v
        return None
```

Rehashing: when α exceeds threshold (e.g. 0.75), allocate $2m$ slots and reinsert all keys: $O(n)$ but amortised $O(1)$.

6.3 Open Addressing

All keys live in T itself. Probe sequence $h(k,0), h(k,1), \ldots, h(k,m-1)$ is a permutation of slots.

- **Linear probing:** $h(k,i) = (h(k) + i) \bmod m$. Suffers *primary clustering* (long runs).
- **Quadratic probing:** $h(k,i) = (h(k) + c_1 i + c_2 i^2) \bmod m$. Reduces primary clustering.
- **Double hashing:** $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. Best distribution; $h_2(k) \neq 0$.

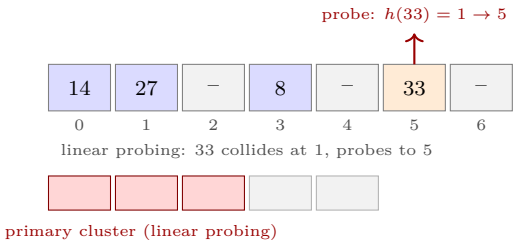

Figure 6.3: Open addressing with linear probing. Collisions probe forward; clusters grow and degrade performance.

Load factor $\alpha = n/m < 1$ (must keep free slots). Expected probes $\approx 1/(1-\alpha)$ for unsuccessful search with uniform hashing. Deletion cannot just clear a slot (breaks probe chains) — mark as `DELETED`/tombstone or rehash.

```python
# Open addressing -- linear probing
class OpenHash:
    EMPTY, DELETED = object(), object()
    def __init__(self, m=8):
        self.m, self.T = m, [self.EMPTY]*m
    def _h(self, k, i): return (hash(k) + i) % self.m
    def insert(self, k):
        for i in range(self.m):
            j = self._h(k, i)
            if self.T[j] in (self.EMPTY, self.DELETED):
                self.T[j] = k; return j
        raise RuntimeError("table full")
    def search(self, k):
        for i in range(self.m):
            j = self._h(k, i)
            if self.T[j] is self.EMPTY: return None
            if self.T[j] == k: return j
        return None
```

6.4 Comparison

6.5 Chapter Notes

CLRS Ch. 11, Weiss Ch. 5. Python `dict`/`set` use open addressing; Java `HashMap` uses chaining with treeification.

Aspect	Chaining	Open addressing
α can exceed 1	yes	no ($\alpha < 1$)
Deletion	easy	tombstones
Cache locality	poor (lists)	excellent
Worst-case search	$\Theta(n)$	$\Theta(m)$

Table 6.1: Chaining vs. open addressing trade-offs.

6.6 Exercises

E6.1 **Chaining.** $m = 5$, $h(k) = k \bmod 5$, insert $14, 27, 8, 33, 91$ with chaining. Draw table and compute expected search cost for present vs. absent keys.

E6.2 **Linear probing.** Same keys/m with linear probing. Show probe sequences and final table. Count total probes for all insertions.

E6.3 **Universal bound.** For universal family $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$, prove $\Pr[h(k) = h(\ell)] \leq 1/m$ for $k \neq \ell$. Deduce expected chain length $\leq 1 + \alpha$.

E6.4 **Tombstones.** Show that naively clearing a slot on deletion in open addressing breaks search. Give an example and explain why tombstones fix it but degrade performance over many deletions.

E6.5 **Load factor.** With $\alpha = 0.9$ and uniform hashing, estimate expected probes for unsuccessful search in open addressing via $1/(1 - \alpha)$. At what α does rehashing become worthwhile?

Chapter 7

Graphs: Representations, Traversals & Preview

> *"A graph is just vertices and edges."* — Yet it models networks, dependencies, maps, and the web itself.

Prerequisites: Ch. 2 (queues/stacks), Ch. 3 (BFS/DFS intuition), Ch. 5 (priority queue for Dijkstra/MST preview).

Learning Outcomes

After this chapter you will be able to:

(L1) represent graphs via adjacency matrix vs. adjacency list and compare costs;

(L2) execute BFS and DFS, classify edges, and analyse $O(n + m)$ time;

(L3) preview MST (Kruskal/Prim) and Dijkstra's shortest paths at a high level;

(L4) model a problem as a graph and choose the right representation.

7.1 Graph Basics & Representations

A graph $G = (V, E)$ has $|V| = n$ vertices and $|E| = m$ edges. Directed vs. undirected; weighted vs. unweighted. Degree $\deg(v)$ counts incident edges.

Two canonical representations:

Implementation: matrix as `vector<vector<int>>` or bitset; list as `vector<vector<int>>` (weighted variant uses `pair<int,int>` for (v, w)).

```
# Adjacency list -- Python
n = 4
```

Operation	Adj. matrix $n \times n$	Adj. list
Space	$\Theta(n^2)$	$\Theta(n+m)$
Edge (u, v) exists?	$O(1)$	$O(\deg(u))$
List neighbours of u	$\Theta(n)$ scan	$\Theta(\deg(u))$
Sparse $m \ll n^2$	wasteful	efficient
Dense $m \approx n^2$	fine	overhead

Table 7.1: Adjacency matrix vs. list. Choose list for sparse graphs (most real networks).

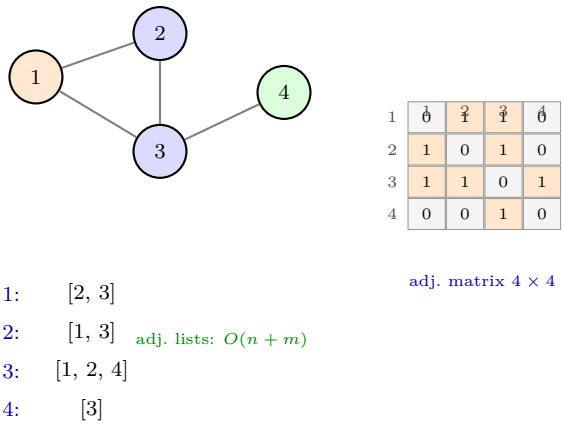

Figure 7.1: Undirected graph with 4 vertices, 4 edges: adjacency matrix (symmetric, orange = 1) vs. adjacency lists.

```
adj = [[] for _ in range(n)]
def add_edge(u, v, w=1):  # 0-indexed, undirected
    adj[u].append((v, w))
    adj[v].append((u, w))
add_edge(0,1); add_edge(0,2); add_edge(1,2); add_edge(2,3)
# adj[2] == [(0,1),(1,1),(3,1)]
```

7.2 BFS & DFS

Both traverse in $O(n+m)$ with adjacency lists ($O(n^2)$ with matrix).

BFS uses a queue: explores in increasing distance from source. Yields shortest paths (fewest edges) in unweighted graphs. Records `dist` and `parent`.

DFS uses a stack (recursion): goes deep before backtracking. Classifies edges as tree/back/forward/cross (directed case). Useful for topological sort and cycle detection.

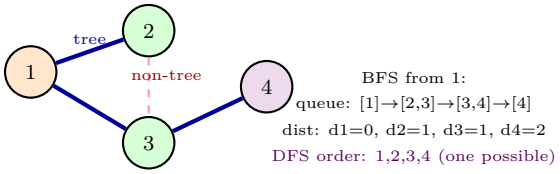

Figure 7.2: BFS tree from vertex 1 (blue thick = tree edges). Dashed edge (2,3) is non-tree; BFS finds shortest-path distances.

```
from collections import deque
```

```python
def bfs(adj, s):
    n = len(adj)
    dist = [-1]*n; parent = [-1]*n
    dist[s]=0; q=deque([s])
    order=[]
    while q:
        u=q.popleft(); order.append(u)
        for v,_ in adj[u]:
            if dist[v]==-1:
                dist[v]=dist[u]+1; parent[v]=u; q.append(v)
    return order, dist, parent

def dfs(adj, s, visited=None, order=None):
    if visited is None: visited=set(); order=[]
    visited.add(s); order.append(s)
    for v,_ in adj[s]:
        if v not in visited: dfs(adj, v, visited, order)
    return order
```

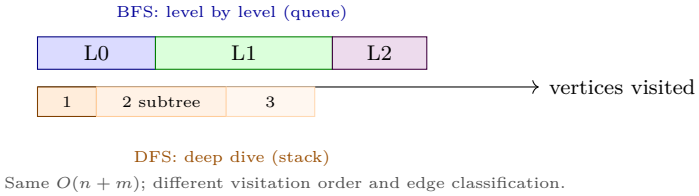

Figure 7.3: BFS vs. DFS visitation pattern on the same graph. BFS is level-order; DFS plunges deep.

7.3 Preview: MST & Shortest Paths

Minimum Spanning Tree (MST): cheapest set of $n-1$ edges connecting all vertices. Kruskal sorts edges and union-finds; Prim grows via min-heap (Ch. 5): both $O(m \log n)$.

Dijkstra: single-source shortest paths for non-negative weights. Repeatedly extracts min-distance vertex from a heap and relaxes edges: $O((n+m)\log n)$ with binary heap. BFS is the special case $w = 1$.

These are developed fully in SC2001; here we note that the right heap + graph representation gives the right bound.

7.4 Chapter Notes

CLRS Ch. 20–24, Weiss Ch. 9. Adj. list is default for sparse graphs; matrix for dense or fast edge queries. BFS/DFS are the workhorses for connectivity, bipartiteness, and topological sort.

7.5 Exercises

E7.1 **Representations.** For $n = 1000$, $m = 3000$, compute space for matrix (1 byte/entry) vs. list (8 bytes/edge endpoint). At what m does matrix win?

E7.2 **BFS distances.** Run BFS from vertex 1 on Fig. 7.1. Give `dist`, `parent`, and BFS tree. Prove BFS indeed gives shortest-path distances in unweighted graphs.

E7.3 **DFS classification.** Run DFS from 1 on the same graph (neighbour order increasing). Classify each edge as tree/back/forward/cross (treat undirected edges as two directed arcs).

E7.4 **Dijkstra preview.** Add weights $w(1,2) = 4, w(1,3) = 2, w(2,3) = 1, w(3,4) = 5$. Run Dijkstra from 1 by hand with a heap; show extract order and final distances. Why does negative weight break it?

E7.5 **MST preview.** Give MST of the weighted graph above (list edges and total weight). Run Kruskal and Prim and compare steps.

Chapter 8

Sorting & Asymptotic Analysis

"Sorting is the most studied problem in computing." — And the sharpest lens for $O(\cdot)$ reasoning.

Prerequisites: Ch. 1–7 (arrays, heaps, recurrences, $O(\cdot)$). This chapter ties SC1007 together: classic sorts, linear-time sorts, lower bounds, and recurrence preview.

Learning Outcomes

After this chapter you will be able to:

(L1) implement and analyse quicksort, merge sort, and heap sort;

(L2) apply counting sort and radix sort for $O(n)$ sorting under assumptions;

(L3) prove the $\Omega(n \log n)$ comparison lower bound via decision trees;

(L4) solve recurrences and compare $O(n \log n)$ vs. $O(n^2)$ vs. $O(n)$.

8.1 Comparison Sorts: Quick, Merge, Heap

All comparison sorts must be $\Omega(n \log n)$ worst-case (decision-tree bound, Sec. 8.3). Three $O(n \log n)$ champions:

Merge sort: divide at mid, recurse, merge $\Theta(n)$. Recurrence $T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$ worst-case. Stable, not in-place (needs $\Theta(n)$ auxiliary).

Heap sort: build max-heap $O(n)$, repeatedly extract max $O(n \log n)$ in-place, not stable.

Quicksort: partition around pivot $\Theta(n)$, recurse on sides. Average $\Theta(n \log n)$; worst $\Theta(n^2)$ (bad pivots) but randomised pivot gives expected $\Theta(n \log n)$. In-place, fast in practice.

```
# Quicksort -- Lomuto partition, randomised pivot
import random
def partition(a, lo, hi):
    pivot = a[hi]
```

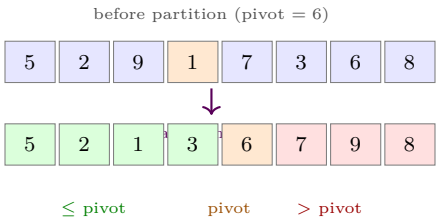

Figure 8.1: Quicksort partition: pivot 6 moves to final position; left $\leq$ pivot, right $>$ pivot. Recurse on both sides.

```
    i = lo
    for j in range(lo, hi):
        if a[j] <= pivot:
            a[i], a[j] = a[j], a[i]; i += 1
    a[i], a[hi] = a[hi], a[i]
    return i

def quicksort(a, lo=0, hi=None):
    if hi is None: hi = len(a)-1
    if lo < hi:
        # random pivot for expected O(n log n)
        r = random.randint(lo, hi); a[r], a[hi] = a[hi], a[r]
        p = partition(a, lo, hi)
        quicksort(a, lo, p-1); quicksort(a, p+1, hi)
```

```
# Merge sort -- stable, O(n log n) worst-case
def merge(a, lo, mid, hi, tmp):
    i, j, k = lo, mid+1, lo
    while i <= mid and j <= hi:
        if a[i] <= a[j]: tmp[k]=a[i]; i+=1
        else: tmp[k]=a[j]; j+=1
        k+=1
    while i <= mid: tmp[k]=a[i]; i+=1; k+=1
    while j <= hi: tmp[k]=a[j]; j+=1; k+=1
    a[lo:hi+1]=tmp[lo:hi+1]

def mergesort(a, lo=0, hi=None, tmp=None):
    if hi is None: hi=len(a)-1; tmp=[0]*len(a)
    if lo < hi:
        mid=(lo+hi)//2
        mergesort(a,lo,mid,tmp); mergesort(a,mid+1,hi,tmp)
        merge(a,lo,mid,hi,tmp)
```

8.2 Linear-Time Sorts

When keys have structure, we beat $n \log n$.

Counting sort: for integers in $[0, k]$, count frequencies, prefix sums give positions. Time $\Theta(n + k)$, space $\Theta(k)$. Stable.

Radix sort: sort by digit from LSD to MSD using stable counting sort per digit. For d digits base b:

$O(d(n+b))$.

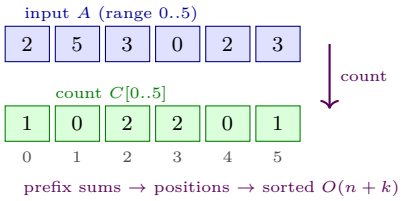

Figure 8.2: Counting sort: histogram C then prefix sums place each element directly. Radix sort repeats this per digit.

```python
# Counting sort -- O(n+k), stable
def counting_sort(a, k):
    c = [0]*(k+1)
    for x in a: c[x] += 1
    for i in range(1, k+1): c[i] += c[i-1]
    out = [0]*len(a)
    for x in reversed(a):  # reverse for stability
        c[x]-=1; out[c[x]]=x
    return out

# Radix sort -- LSD first, stable per digit
def radix_sort(a, base=10):
    maxv = max(a) if a else 0
    exp = 1
    while maxv // exp > 0:
        # counting sort by digit (exp place)
        buckets = [[] for _ in range(base)]
        for x in a: buckets[(x//exp) % base].append(x)
        a = [x for b in buckets for x in b]
        exp *= base
    return a
```

8.3 Lower Bound & Analysis

Theorem 8.1 (Comparison lower bound)**.** Any comparison sort needs $\Omega(n \log n)$ comparisons worst-case.

Sketch. Decision tree has $n!$ leaves (one per permutation). Binary tree with L leaves has height $\geq \lceil \log_2 L \rceil$. So height $\geq \log_2(n!) = \Theta(n \log n)$ by Stirling. □

Recurrence preview: merge sort $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$; quicksort average $T(n) = T(k) + T(n-k-1) + \Theta(n)$ with $k \sim$ Uniform gives $\Theta(n \log n)$ expected.

8.4 Stability, Hybrids & Recurrence Details

Stability. A sort is *stable* if equal keys retain input order. Merge and counting/radix are stable; heap and quicksort (as usually implemented) are not. Stability matters when sorting by multiple keys.

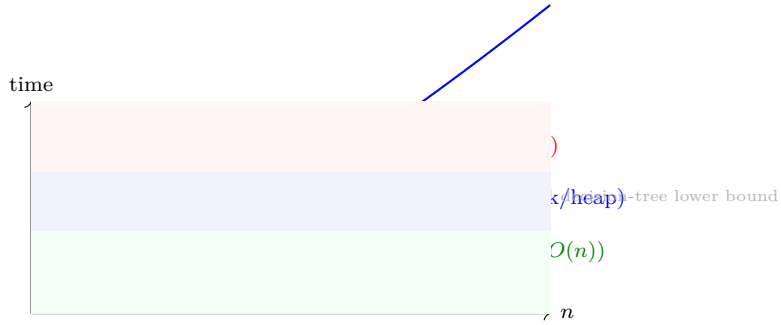

Figure 8.3: Sorting landscape: comparison sorts cannot beat $n \log n$; counting/radix break the bound by using key structure.

Introsort (STL `sort`) runs quicksort but switches to heapsort when recursion depth exceeds $2 \log n$, guaranteeing $O(n \log n)$ worst-case while keeping quicksort's speed. Small subarrays (< 16) use insertion sort.

Recurrence bridge to SC2001. Merge sort gives $T(n) = 2T(n/2) + \Theta(n)$. Master theorem (SC2001 Ch. 3) yields $\Theta(n \log n)$. Quicksort average case uses indicator variables: expected comparisons $= 2 \sum_{i<j} 1/(j-i+1) = \Theta(n \log n)$.

```
# Introsort sketch -- depth-limited quicksort
import heapq
def introsort(a, lo, hi, depth_limit):
    if hi - lo < 16:
        insertion_sort(a, lo, hi); return
    if depth_limit == 0:
        heapsort_range(a, lo, hi); return
    p = partition(a, lo, hi)
    introsort(a, lo, p-1, depth_limit-1)
    introsort(a, p+1, hi, depth_limit-1)
```

8.5 Chapter Notes

CLRS Ch. 2, 6–8, Weiss Ch. 7. Quicksort is usually fastest in practice despite worst case; introsort (STL `sort`) switches to heapsort when recursion deepens.

8.6 Exercises

E8.1 **Quicksort trace.** Sort $[5, 2, 9, 1, 7, 3, 6, 8]$ with quicksort, pivot = last element. Show partitions and recursion tree. Count comparisons.

E8.2 **Merge analysis.** Prove $T(n) = 2T(n/2) + n = \Theta(n \log n)$ by recursion tree. Is merge sort stable? Is it in-place? Justify.

E8.3 **Counting sort.** Sort $[2, 5, 3, 0, 2, 3]$ with counting sort ($k = 5$). Show C, prefix sums, and output construction. Why does iterating reversed preserve stability?

E8.4 **Lower bound.** Prove $\log_2(n!) = \Theta(n \log n)$ using Stirling or integral bounds. Explain why counting sort

does not violate the comparison lower bound.

E8.5 **Hybrids.** Describe introsort (quicksort → heapsort fallback + insertion sort for small n). Why does STL use it? Analyse its $O(n \log n)$ worst-case guarantee.